

Module 07: Communication — Multi-Agent Simulation and Signaling Games

Learning Objectives

1. Build a multi-agent simulation where agents observe and influence each other.
2. Implement a signaling game with sender and receiver Active Inference agents.
3. Track mutual information to measure emergent communication.

Introduction

Active Inference agents don't exist in isolation. When multiple agents share an environment, each agent's actions become part of the other agents' observations. This creates a rich dynamics: agents develop implicit communication by learning to predict and influence each other's behavior. This module builds multi-agent simulations from the single-agent tools developed in Modules 01–06.

Key Concepts

1. Multi-Agent Architecture

In a multi-agent setup, each agent has its own generative model, but the environment's state includes the states and actions of all agents:

```

from active_inference.agent import GenerativeModel, ActiveInferenceAgent,

# Agent 1: Sender
model_sender = GenerativeModel(A=A_sender, B=B_sender, C=C_sender, D=D_sender)
agent_sender = ActiveInferenceAgent(model_sender, gamma=4.0)

# Agent 2: Receiver
model_receiver = GenerativeModel(A=A_receiver, B=B_receiver, C=C_receiver, D=D_receiver)
agent_receiver = ActiveInferenceAgent(model_receiver, gamma=4.0)

```

2. The Signaling Game

A canonical test for emergent communication:

- **2 world states:** food-left (0), food-right (1)
- **Sender:** observes the true state, selects a signal (action 0 or 1)
- **Receiver:** observes the sender's signal, selects a direction (go-left or go-right)
- **Success:** the receiver reaches the food

The sender's A-matrix allows it to observe the world state. The receiver's A-matrix maps signals to observations. Reward is shared — both agents prefer the receiver reaching the food.

```

# Sender's A-matrix: directly observes the true state
A_sender = np.eye(2)  # o = s (fully observable)

# Receiver's A-matrix: observes the sender's action
A_receiver = np.eye(2)  # initially: signal 0 → obs 0, signal 1 → obs 1

```

3. Multi-Agent Simulation Loop

```
num_steps = 100
mi_history = []

for t in range(num_steps):
    # Sender observes world state and produces a signal
    obs_sender = env.get_observation(agent_id=0)
    signal = agent_sender.step(obs_sender)

    # Receiver observes the signal and selects an action
    obs_receiver = signal # receiver sees sender's action
    direction = agent_receiver.step(obs_receiver)

    # Environment evaluates: did receiver find food?
    reward = env.evaluate(direction)

    # Track mutual information between signals and world states
    mi = compute_mutual_information(signals, states)
    mi_history.append(mi)
```

4. Measuring Communication with Mutual Information

Mutual information $I(X; Y)$ quantifies how much the sender's signals reduce uncertainty about the world state:

$$I(\text{signal} ; \text{state}) = H(\text{signal}) + H(\text{state}) - H(\text{signal}, \text{state})$$

If $I = 0$, signals are uncorrelated with states (no communication). If $I = H(\text{state})$, signals perfectly encode the state.

```

from active_inference.math import mutual_information

# Build a joint distribution from observed signal-state pairs
joint = np.zeros((2, 2))
for s, sig in zip(world_states, signals):
    joint[sig, s] += 1
joint /= joint.sum()

mi = mutual_information(joint)
print(f"Mutual Information: {mi:.4f} bits")

```

5. Learning to Communicate

Neither agent has a built-in communication protocol. Communication **emerges** through learning:

1. The sender learns (via pA/pB updates) which signals lead to reward.
2. The receiver learns which signals correlate with which world states.
3. Over time, mutual information increases as the agents develop a shared code.

6. Scaling to More Agents

The same pattern extends to N agents:

- Each agent maintains its own `GenerativeModel` and `ActiveInferenceAgent`
- The environment maps agent actions to other agents' observations
- Mutual information can be tracked pairwise

Applications

- **Language evolution:** How do symbolic systems emerge from non-linguistic agents?
- **Cooperative robotics:** Agents that develop coordination protocols without pre-programmed signals.

- **Social inference:** An agent that models other agents' beliefs (theory of mind) as part of its generative model.

Conclusion

Multi-agent Active Inference extends the single-agent framework naturally: each agent's actions become observations for other agents. Communication emerges when agents learn that their signals systematically influence others' behavior. Module 08 extends the temporal dimension — planning over multiple future steps.